allowed in the `#{...}` construct. Here we invoked the `capitalize` method, defined for all strings, to output our parameter with a leading uppercase letter.

Strings are a fairly primitive data type that contain an ordered collection of bytes or characters. Ruby also provides means for defining collections of arbitrary objects via *arrays* and *hashes*.

Arrays and Hashes

Ruby's arrays and hashes are indexed collections. Both store collections of objects, accessible using a key. With arrays, the key is an integer, whereas hashes support any object as a key. Both arrays and hashes grow as needed to hold new elements. It's more efficient to access array elements, but hashes provide more flexibility. Any particular array or hash can hold objects of differing types; you can have an array containing an integer, a string, and a floating-point number, for example.

You can create and initialize a new array object by using an *array literal*—a set of elements between square brackets. Given an array object, you can access individual elements by supplying an index between square brackets, as the next example shows. Ruby array indices start at zero:

```
a = [ 1, "cat", 3.14 ]   # array with three elements
a[0]                     # access the first element (1)
a[2] = nil               # set the third element
                         # array now [ 1, "cat", nil ]
```

You may have noticed that we used the special value `nil` in this example. In many languages, the concept of *nil* (or *null*) means *no object.* In Ruby, that's not the case; `nil` is an object, like any other, that happens to represent nothing.

The `<<` method is often used with arrays. It appends a single value to its receiver: